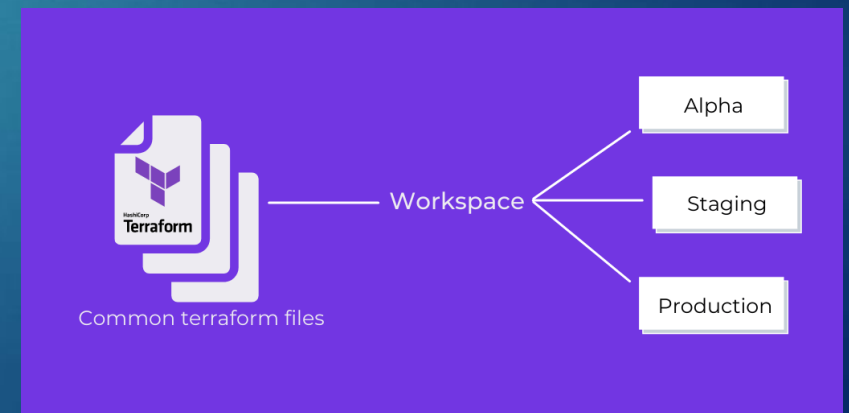
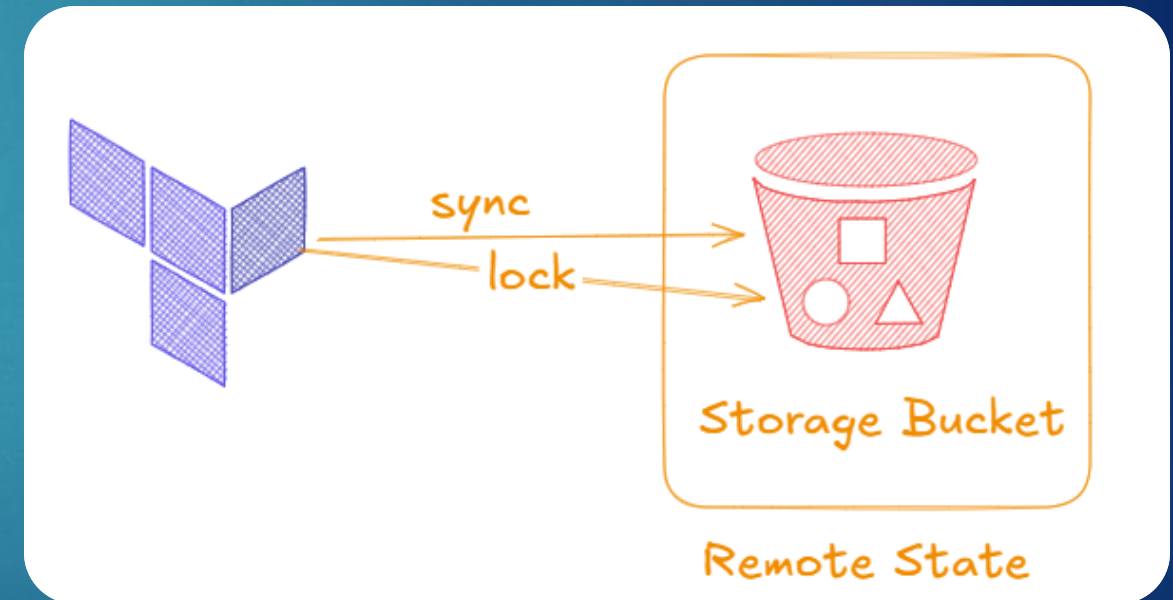


- ▶ A .tfvars file is used to hardcode configuration values directly into your Terraform source files
 - ▶ True
 - ▶ False ☒
- ▶ Which best describes the purpose of marking a Terraform variable as sensitive?
 - ▶ It makes the variable immutable
 - ▶ It prevents the variable from being modified by other modules
 - ▶ It hides the variable value from CLI output and logs ☒
 - ▶ It adds encryption to the source code
- ▶ What is the recommended way to manage different values (e.g., instance types) across dev, test, and prod environments?
 - ▶ Hardcode them in main.tf
 - ▶ Use terraform destroy between environments
 - ▶ Use .tfvars files and workspaces ☒
 - ▶ Manually edit the state file

- ▶ Challenges with Terraform: Code reuse and modularization
- ▶ Modules:
 - ▶ Logical containers for grouping related resources
 - ▶ Enable code reuse across projects and environments
 - ▶ Example: VPC or database modules for consistent infrastructure
- ▶ Workspaces:
 - ▶ Manage multiple environments (e.g., dev, prod) using the same configuration
 - ▶ Keep state isolated per environment
 - ▶ Simplified workflow with commands like `terraform workspace new dev`



- ▶ Challenges with Terraform: State Management
- ▶ Local State
 - ▶ Stored on the developer's machine
 - ▶ Suitable for single-user setups
- ▶ Remote State
 - ▶ Stored in shared backends in S3 for collaboration and locking



- ▶ Challenges with Terraform: Securing Sensitive Data
 - ▶ Avoid hardcoding sensitive data in .tf files or state files
 - ▶ Use sensitive = true for variables to prevent exposing data in logs
 - ▶ Store secrets in secure solutions like AWS Secrets Manager or GitHub Secrets (more later)



DevOps & Pipelines

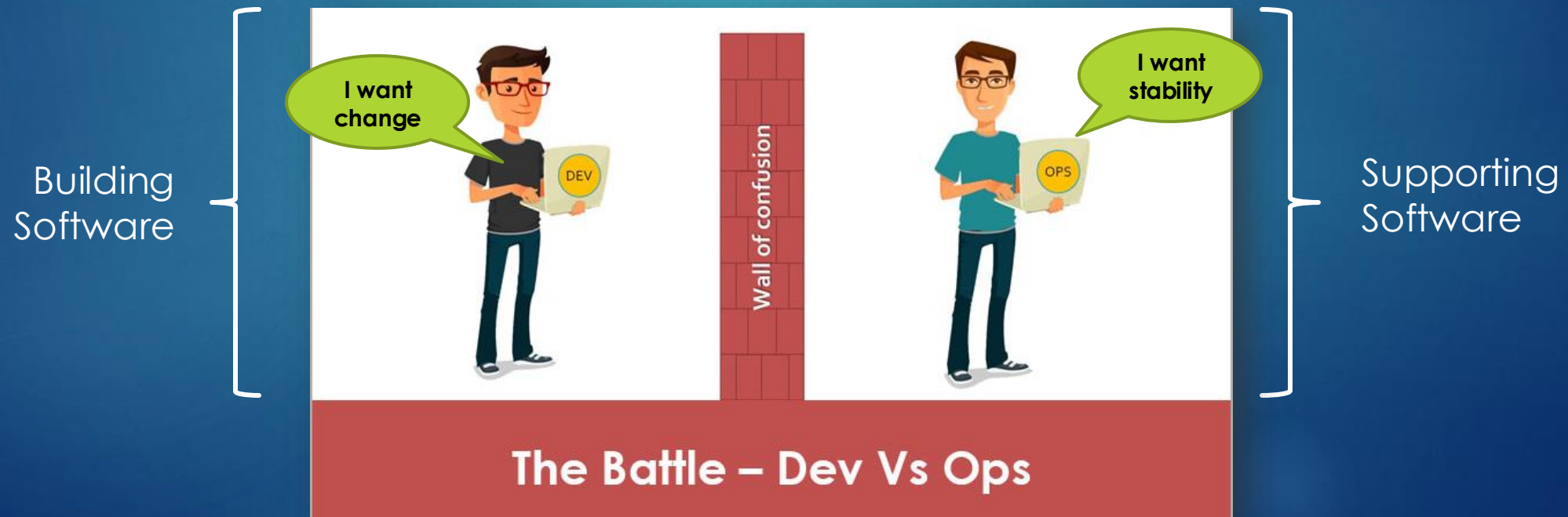
Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



A Very Common Challenge in IT

- ▶ Deployments to production happening very rarely (only a few times a year)
- ▶ Testing happens months after the development finishes
- ▶ Operations team (which supports production) is constantly fire-fighting
- ▶ Developers find it near impossible to get any IT changes done
- ▶ Question: Could the Development Methodology be part of the problem?



Waterfall Methodology

- ▶ Takes a methodical, sequential approach to developing software
- ▶ It starts with extensive planning and design and then the development team codes the software
- ▶ When the development team is finished, another team tests it, and once testing is complete the software is handed off to operations to deploy
- ▶ Requirements can (and will) change in the middle of the dev process, which Waterfall doesn't take this into consideration
- ▶ It focuses solely on delivering working software, and it doesn't consider how the software will work in a production environment



Agile Methodology

- ▶ While Waterfall doesn't work well with change, Agile does because the focus is placed on delivering small iterations of software instead of a whole product
- ▶ This makes it much easier to adjust the software when requirements change
- ▶ An Agile team is cross-functional and is made up of one team member from planning, another from coding, another from testing and so on
- ▶ Like Waterfall, Agile also focuses on just creating working software, and it doesn't consider how the live production environment will affect the software and vice versa, how the software will impact the live environment

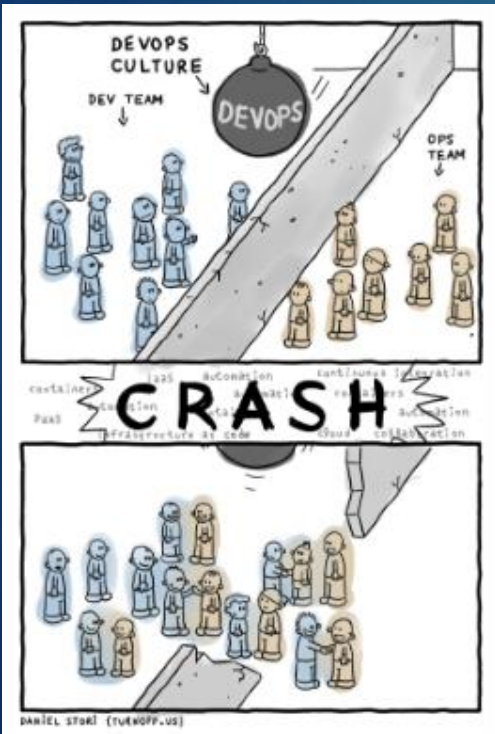


What needs the Change?

- ▶ Both methodologies exclude the Operations team in the software development process
- ▶ This leads to problems and discord between the development team and the Operations team

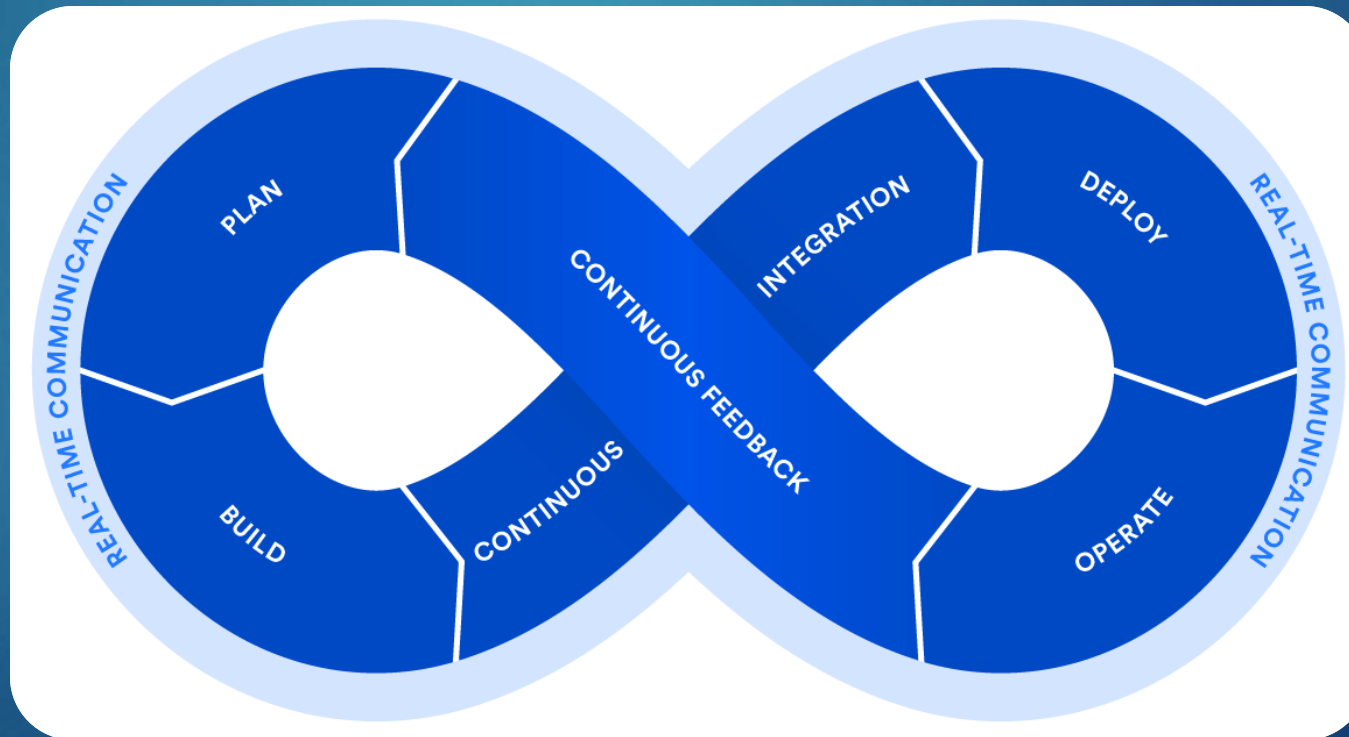
Why?

- ▶ When something goes wrong, it's hard to tell if the problem lies in the code or in the deployment
- ▶ This results in a lot of blame between the two departments due to the “wall” between development and operations
- ▶ A new approach emerged called DevOps, which removes the “wall” between the Development and Operation teams



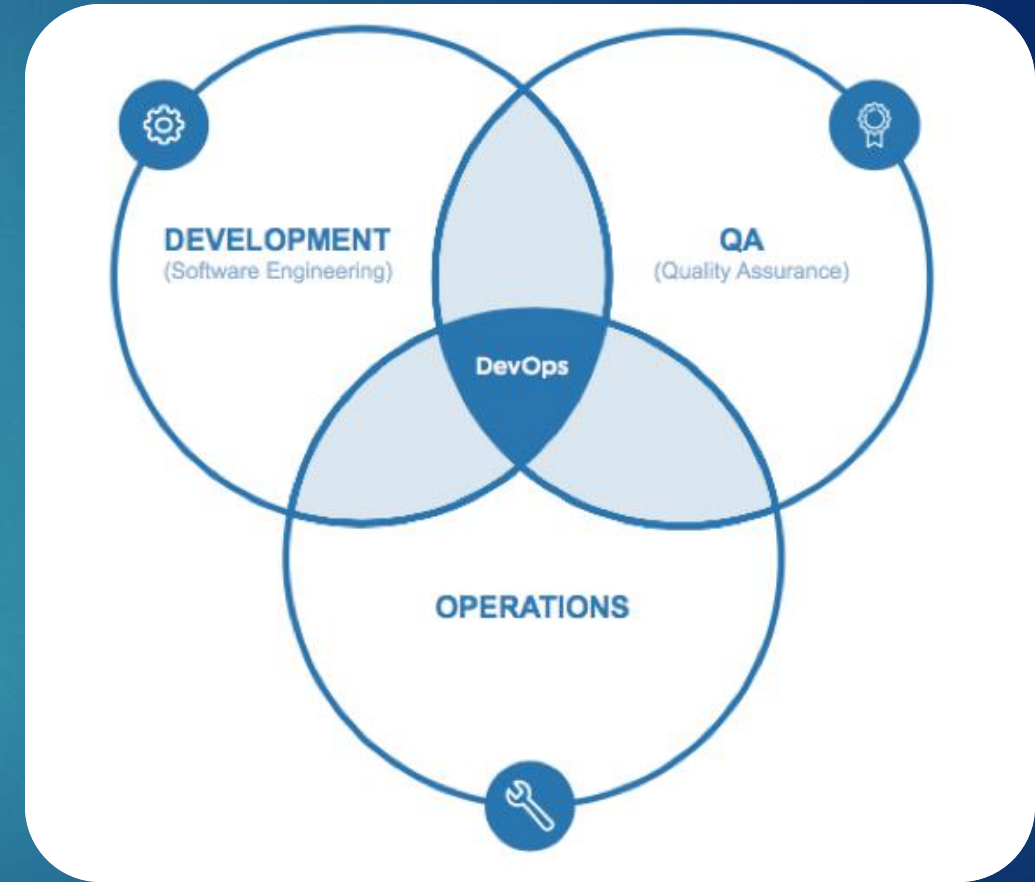
DevOps - Defined

- ▶ DevOps is the combination of cultural philosophies, practices, and tools that increases an organization's ability to deliver applications and services at high velocity: evolving and improving products at a faster pace than organizations using traditional software development and infrastructure management processes



Why is DevOps important?

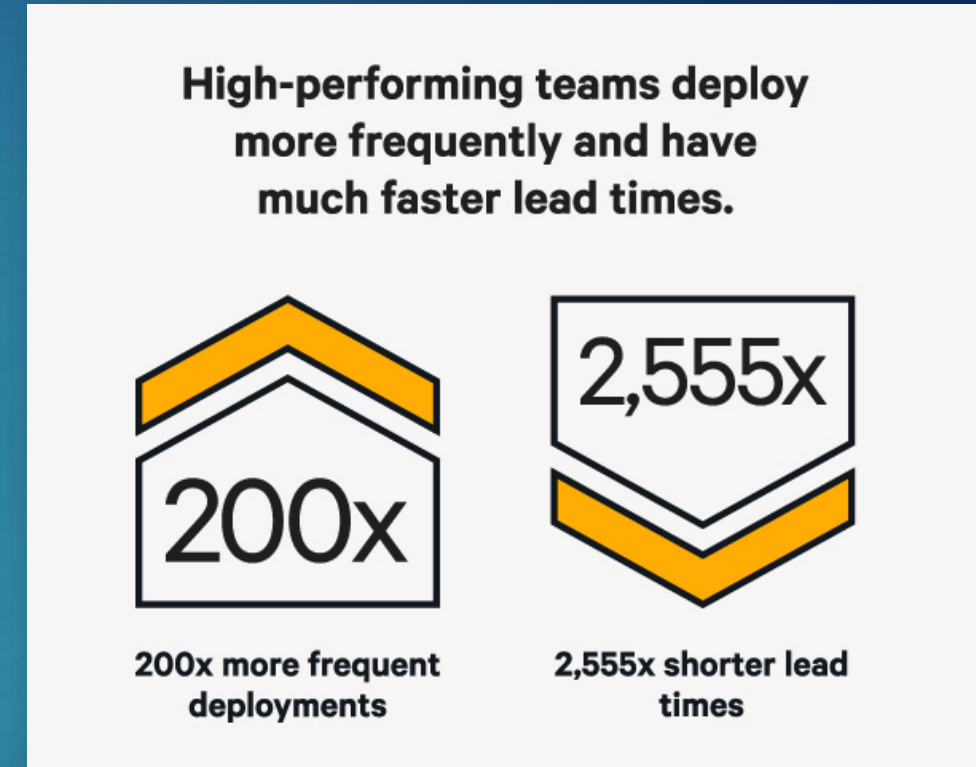
1. Shorter Development Cycles, Faster Innovation
2. Reduced Deployment Failures, Rollbacks, and Time to Recover
3. Improved Communication and Collaboration
4. Increased Efficiencies through Automation



Shorter Development Cycles, Faster Innovation

12

- ▶ When development and operations teams work separately, it's difficult to assess if an application is truly ready for production
- ▶ Handoffs between teams slow down deployment cycles, introducing delays and inefficiencies
- ▶ Integrating development and operations streamlines workflows, enabling faster, more reliable delivery of production-ready applications



Reduced Deployment Failures, Rollbacks, and Time to Recover

- ▶ Many deployment failures are caused by coding issues that go undetected until late in the release process
- ▶ DevOps promotes shorter, more frequent development cycles—helping teams catch defects early through continuous integration and testing
- ▶ Rapid iteration and automation reduce the chance of bugs reaching production
- ▶ A strong DevOps culture fosters shared ownership, enabling faster diagnosis and resolution when issues arise
- ▶ Automated testing and deployment pipelines help eliminate manual errors, improving overall release stability

They make changes with fewer failures, and recover faster from failures.



**3x lower
change failure rate**

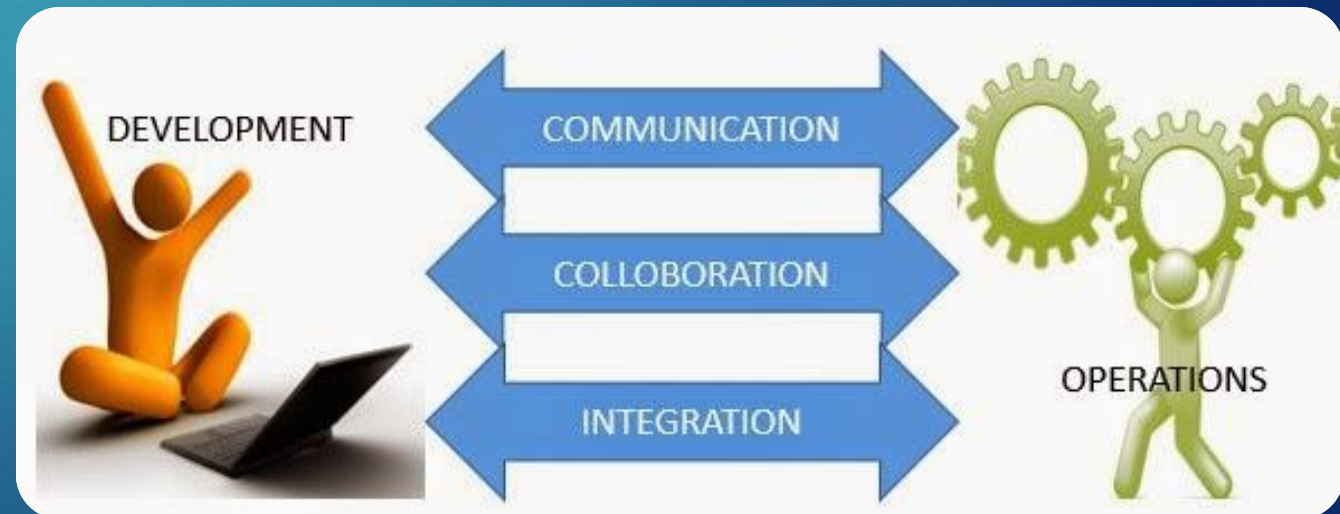


**24x faster
recovery from failures**

Improved Communication and Collaboration

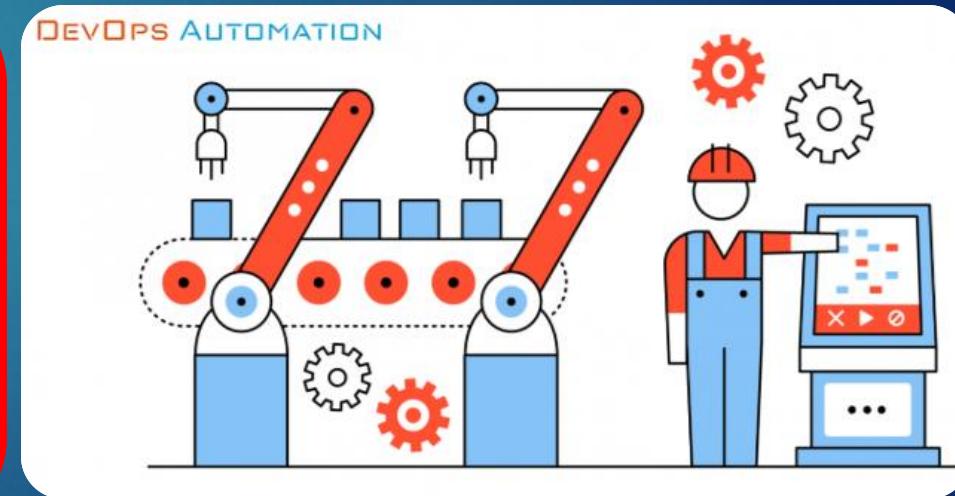
14

- ▶ By breaking down silos, DevOps promotes a shared sense of ownership and accountability between development and operations teams
- ▶ When teams have confidence in each other's expertise, they can take calculated risks, experiment, and iterate more effectively
- ▶ Instead of worrying about handoffs, teams can concentrate on efficiently bringing products to market or into production
- ▶ With everyone aligned toward a common goal, workflows become more efficient, predictable, and scalable



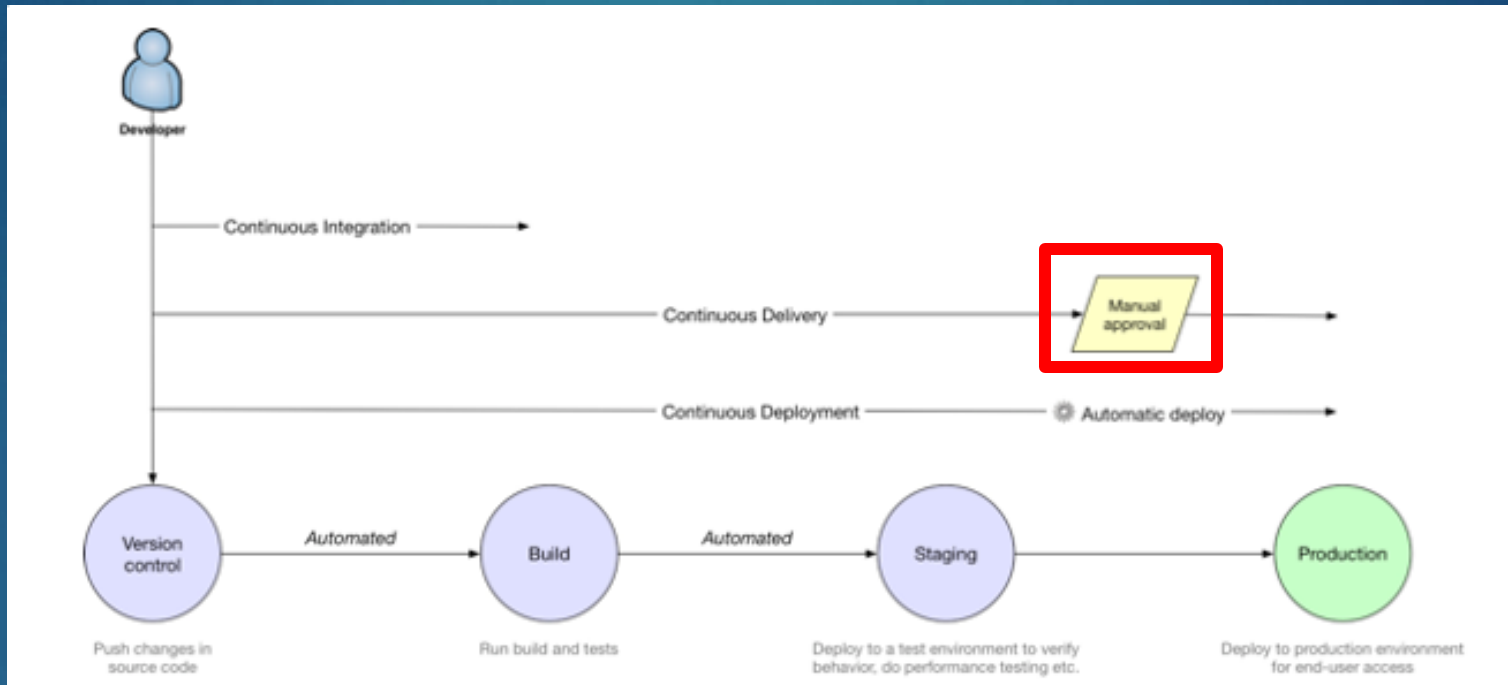
Increased Efficiencies through Automation 15

- ▶ Increased efficiency helps to speed the development process and make it less prone to error. This includes:
 - ▶ **Test Automation** - Test the code and the underlying infrastructure in an *automated* way instead of doing it manually
 - ▶ **Centralized Configuration Management** - If there are multiple running servers and you need to make configuration changes in one file then it is better to *automate* this process rather than do it manually
 - ▶ **Continuous Integration** – Development practice that requires developers to integrate code into a shared repository several times a day. Each check-in is then verified by an *automated* build, allowing teams to detect problems early.
 - ▶ **Continuous Delivery** – A software development practice where code changes are *automatically* prepared for a release to production



Continuous Integration/Continuous Deployment 16

- aka “CI/CD Pipeline”



- ▶ The difference between **Continuous Delivery** and **Continuous Deployment** is the presence of a manual approval to update to production
- ▶ With Continuous Deployment, production happens automatically without manual approval

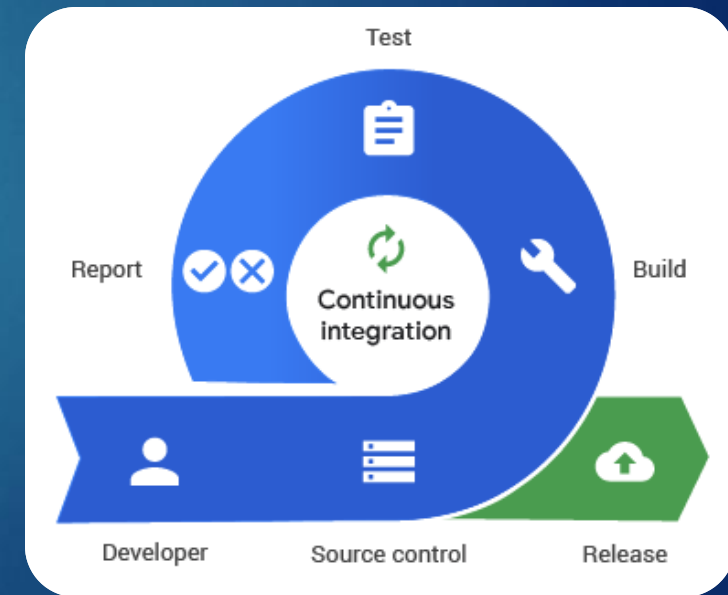
Continuous Integration

► Effort Required

- Your team will need to write automated tests for each new feature, improvement or bug fix
- You need a continuous integration server that can monitor the main repository and run the tests automatically for every new commit pushed
- Developers need to merge their changes as often as possible, at least once a day

► Value Gained

- Less bugs get shipped to production as regressions are captured early by the automated tests
- Building the release is easy as all integration issues have been solved early
- Less context switching as developers are alerted as soon as they break the build and can work on fixing it before they move to another task
- Testing costs are reduced drastically – a CI server can run hundreds of tests in a matter of minutes



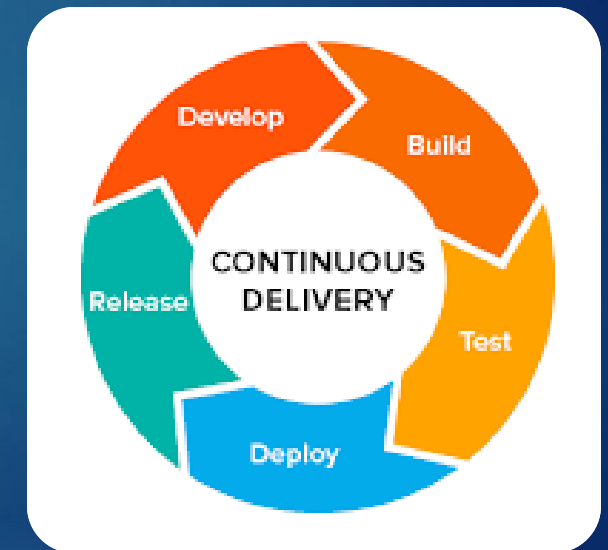
Continuous Delivery

► Effort Required

- You need a strong foundation in continuous integration and your test suite needs to cover enough of your codebase
- Deployments need to be automated. The trigger is still manual but once a deployment is started there shouldn't be a need for human intervention
- Your team will most likely need to embrace feature flags so that incomplete features do not affect customers in production

► Value Gained

- The complexity of deploying software has been taken away and your team doesn't have to spend days preparing for a release anymore
- You can release more often, thus accelerating the feedback loop with your customers
- There is much less pressure on decisions for small changes, hence encouraging iterating faster



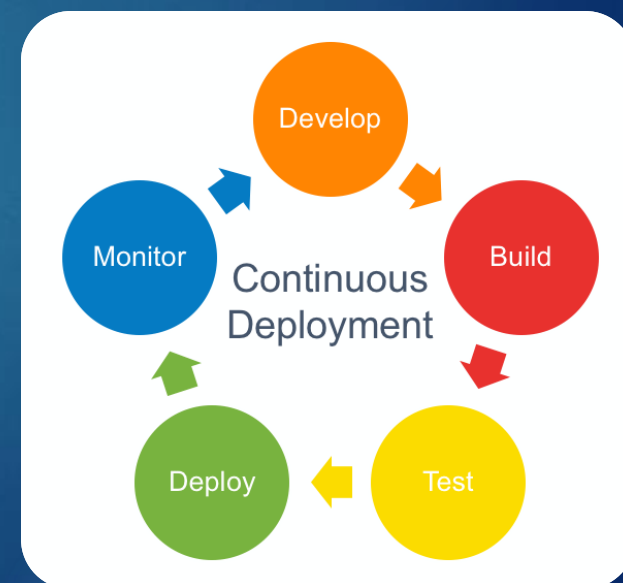
Continuous Deployment

► Effort Required

- Your testing culture needs to be at its best and the quality of your test suite will determine the quality of your releases
- Your documentation process will need to keep up with the pace of deployments
- Feature flags become an inherent part of the process of releasing significant changes to make sure you can coordinate with other departments (Support, Marketing, PR...).

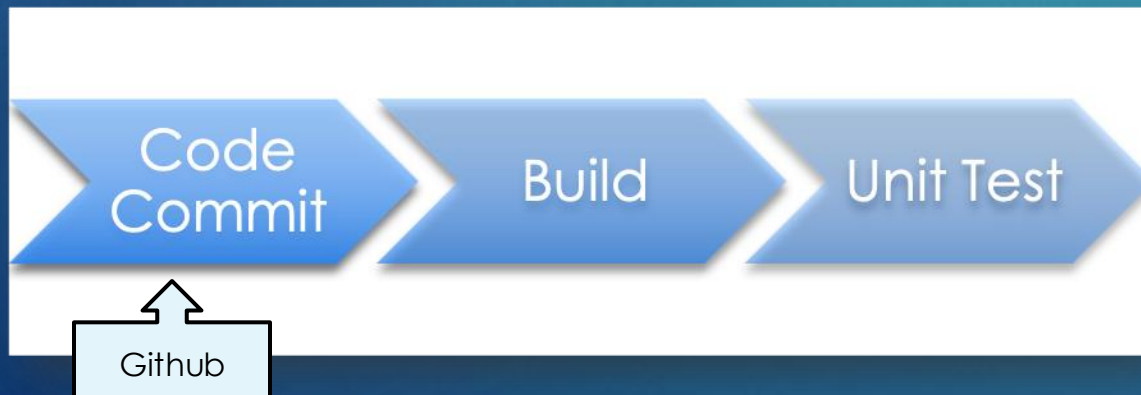
► Value Gained

- You can develop faster as there's no need to pause development for releases. Deployments pipelines are triggered automatically for every change.
- Releases are less risky and easier to fix in case of problem as you deploy small batches of changes
- Customers see a continuous stream of improvements, and quality increases every day, instead of every month, quarter or year



Pipeline - Continuous Integration

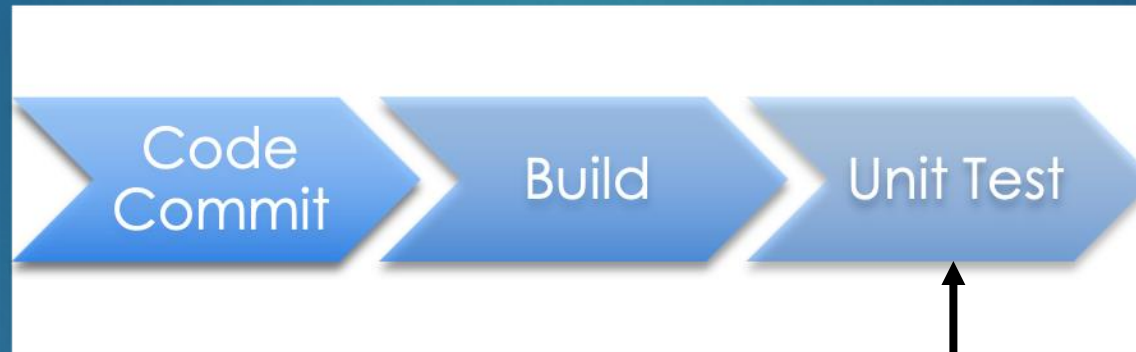
- ▶ Triggered automatically when code is committed to a repository
- ▶ A build process is initiated to ensure code is not broken
- ▶ Unit tests and other validation processes are executed
- ▶ Upon completion, a report is created (success/failed) and/or a deployment process could be initiated to deploy to another environment (test, pre-prod, etc.)



Pipeline - Continuous Integration example

21

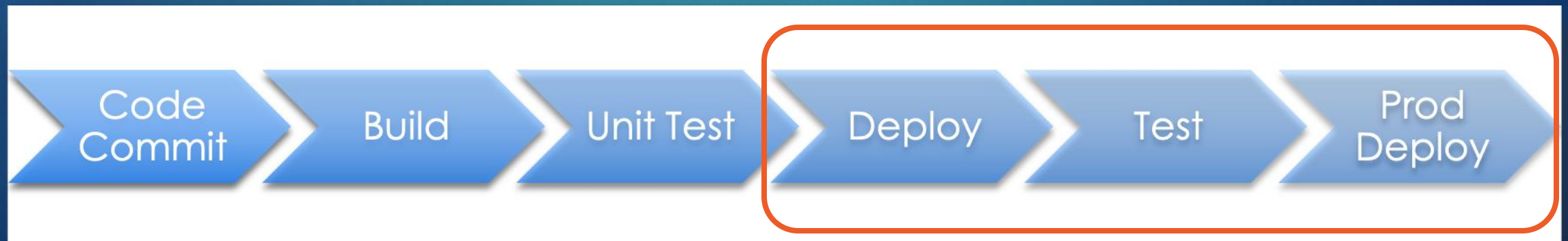
- Several test and validation steps can be executed to ensure code quality



Test/Validation	Benefit
JUnit	Code that tests our code and looks for known regression issues
Jacoco	Measures the amount of app code actually exercised by our unit tests. Requires at least 70% coverage
Aqua	Proactively identify vulnerabilities on open source and license compliance violations, before they manifest in production releases
Checkstyle (Linting)	Validates code matches best practices for formatting to ease maintenance
Veracode	Scans our source code looking for known security and quality issues
SonarQube	Additional code scanning for known security and quality issues

Pipeline - Continuous Deployment/Delivery 22

- ▶ Executes after Continuous Integration steps have successfully completed
- ▶ Application is built and deployed to a pre-prod environment
- ▶ Additional automated testing and/or Acceptance Testing is conducted
- ▶ If successful, the application is then deployed automatically or pending a manual approval to production



Introducing GitHub Actions

- ▶ **GitHub Actions** allows automated workflows directly within GitHub, enabling continuous integration and deployment for repositories
- ▶ Actions are triggered by GitHub events, such as pushes, pull requests, or schedule-based events, making it easy to automate responses to code changes
- ▶ Workflows are defined in YAML files under the `.github/workflows` directory, allowing for transparent and version-controlled automation setup
- ▶ Workflows can run on GitHub-hosted virtual machines or on custom, self-hosted runners, offering flexibility in deployment environments



GitHub Actions

GitHub Actions - Key Concepts

- ▶ **Workflows:** High-level definition of automated processes, triggered by specific GitHub events
- ▶ **Events:** Triggers that start workflows
- ▶ **Jobs:** Sequence of steps that run on a specific runner. Can run sequentially or in parallel.
- ▶ **Steps:** Individual tasks within jobs, such as running commands or actions
- ▶ **Actions:** Reusable pieces of code that perform specific tasks, which can be sourced from the GitHub Marketplace or custom-defined

```
name: CI Pipeline
on:
  push:
    branches: [ main ] # Trigger on pushes to main branch

jobs:
  build:
    runs-on: ubuntu-latest # Use Ubuntu runner

    steps:
      # Check out the repository code
      - uses: actions/checkout@v3

      # Setup Node.js environment
      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

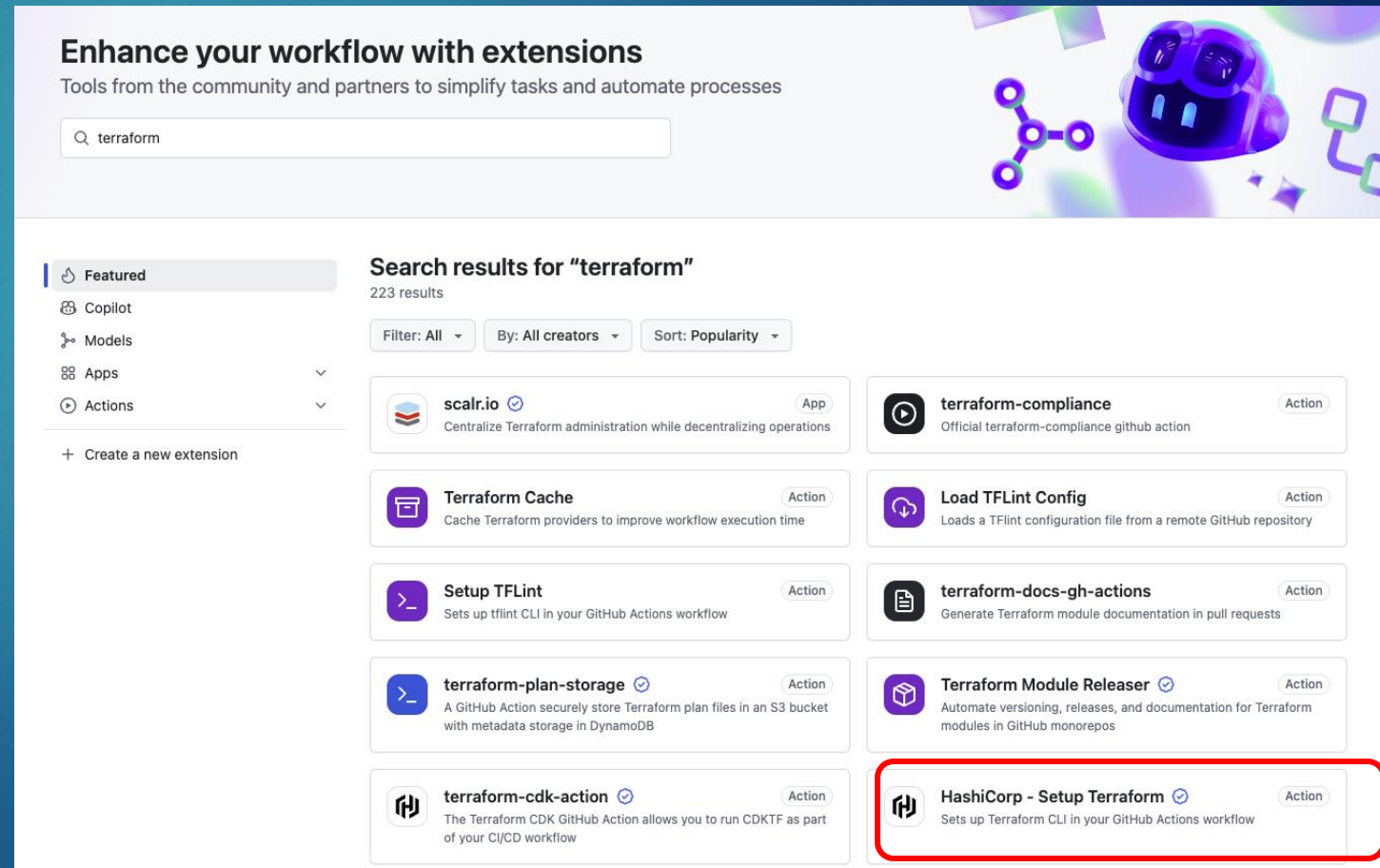
      # Install dependencies (clean install)
      - run: npm ci

      # Run tests
      - run: npm test
```

The workflow triggers on a push event. A single job called "build" that runs on Ubuntu Checks out code, sets up Node.js 18, installs dependencies, and runs tests. Each step runs sequentially and will fail fast if any step fails

GitHub Marketplace

- ▶ GitHub Marketplace is a platform where developers and organizations can discover, purchase, and integrate tools, apps, and services that enhance their workflows within GitHub
- ▶ These offerings include a wide range of integrations for CI/CD, security, project management, testing, and more, aimed at improving productivity and streamlining development processes
- ▶ Many Marketplace offerings are **pre-configured GitHub Actions**, enabling developers to automate workflows directly in their repositories with just a few lines of YAML



GitHub Actions -Terraform

- ▶ After you've used this action, subsequent steps in the same job can run arbitrary Terraform commands using the GitHub Actions **run** syntax
- ▶ This allows most Terraform commands to work exactly like they do on your local command line
- ▶ Using “-auto-approve”, we can run Terraform apply automatically without prompting (assuming no errors)

```
name: Terraform Apply
on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest      # Use Ubuntu runner

    steps:
      - name: Checkout repository
        uses: actions/checkout@v2

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v2.0.2
        with:
          terraform_version: 1.1.9
          terraform_wrapper: false

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan

      - name: Terraform Apply
        run: terraform apply -auto-approve
```

GitHub Runners

- ▶ **GitHub Runners** are the underlying compute instances that execute workflows defined in GitHub Actions
- ▶ They process jobs triggered by events, such as push events or pull requests, in repositories
- ▶ There are 2 types of Runners:
 - ▶ GitHub-Hosted Runners are managed by GitHub, pre-configured with popular tools, and available in various environments (e.g., Linux, Windows, macOS)
 - ▶ Self-Hosted Runners are configurable by users to meet specific requirements, running on their own infrastructure for more control or custom dependencies
- ▶ For most small-to-medium teams, starting with GitHub-hosted runners is convenient
- ▶ Self-hosted runners are a great option for advanced users who want more control or already have robust infrastructure in place



GitHub Environment Variables

- ▶ **GitHub Environment Variables** allow workflows to access dynamic or predefined values that can be used during their execution
- ▶ They provide context about the repository, the workflow itself, or other customizable values, making workflows more flexible and reusable
- ▶ Types of variables:
 - ▶ Predefined GitHub Variables GitHub automatically sets certain environment variables for every workflow Examples: GITHUB_REPOSITORY, GITHUB_REF and GITHUB_EVENT_NAME
 - ▶ Developers can define custom variables in the workflow YAML file using the `env` keyword to store deployment configurations, flags, or other workflow-specific values
 - ▶ Secure variables like API keys or credentials can be defined in GitHub Secrets and accessed securely in workflows. These are encrypted and not visible in logs by default.



GitHub Secrets

- ▶ **GitHub Secrets** Management allows secure storage and usage of sensitive information like API keys, passwords, and credentials in workflows
- ▶ Secrets are encrypted and accessible only within GitHub Actions, protecting sensitive data from exposure in logs or public files
- ▶ Only workflows with explicit permissions can access defined secrets
- ▶ Commonly used for storing credentials, tokens, and configuration values, such as AWS access keys for Terraform



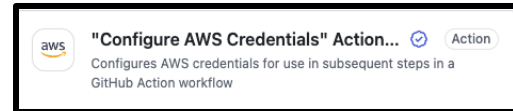
Terraform and Secrets

The screenshot shows the GitHub repository settings for '614test' under the 'Settings' tab. The left sidebar shows the 'Secrets and variables' section expanded, with 'Actions' selected. The main content area is titled 'Actions secrets and variables' and contains two tabs: 'Secrets' and 'Variables'. The 'Variables' tab is active, showing a table of repository variables. The table has three columns: 'Name', 'Value', and 'Last updated'. There is one variable named 'AWS_REGION' with a value of 'us-east-1' and was updated '28 minutes ago'. A 'New repository variable' button is located to the right of the table. Below the table, there is a section for 'Organization variables' with a message stating that organization variables can only be used by public repositories on your plan. A 'Manage organization variables' button is located at the bottom of this section.

Name	Value	Last updated
AWS_REGION	us-east-1	28 minutes ago

```
name: Terraform Apply
on:
push:
branches: [ main ]

jobs:
deploy:
runs-on: ubuntu-latest # Use Ubuntu runner
```



```
steps:
- name: Checkout repository
  uses: actions/checkout@v2

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v2
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: ${ var.AWS_REGION }

- name: Setup Terraform
  uses: hashicorp/setup-terraform@v2.0.2
  with:
    terraform_version: 1.1.9
    terraform_wrapper: false

- name: Terraform Init
  run: terraform init

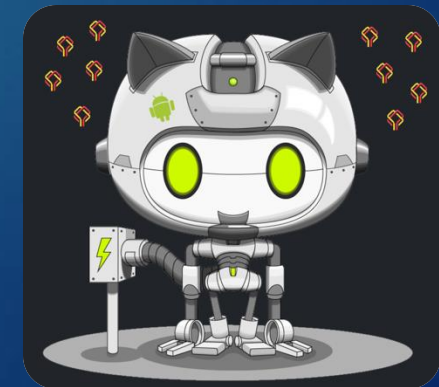
- name: Terraform Plan
  run: terraform plan

- name: Terraform Apply
  run: terraform apply -auto-approve
```

► We are ready to run a Terraform Apply!

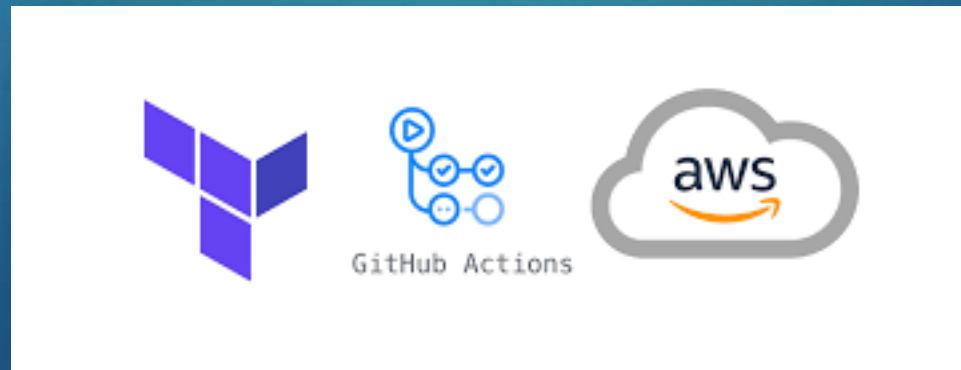
GitHub Actions Best Practices

- ▶ **Leverage Pre-Built Actions** – Utilize the **GitHub Marketplace** to integrate well-tested, reusable actions instead of building custom solutions from scratch, saving time and effort
- ▶ **Secure Sensitive Data with Secrets** – Store confidential information like **API keys, credentials, and tokens** in **GitHub Secrets** to ensure they remain encrypted and protected from exposure
- ▶ **Define Precise Triggers** – Minimize unnecessary workflow executions by carefully configuring **event triggers** (e.g., on: push, on: pull_request) to optimize performance and resource usage
- ▶ **Optimize Workflow Efficiency** – Reduce execution time and improve performance by consolidating or **streamlining steps** and eliminating redundant processes
- ▶ **Pin Action Versions for Stability** – Always specify **fixed action versions** (e.g., actions/checkout@v3) to **prevent breaking changes** and ensure predictable builds
- ▶ **Restrict Access to Self-Hosted Runners** – Limit self-hosted runner access by assigning them **only to specific workflows or repositories**, reducing security risks and unauthorized usage



GitHub Actions and Term Project

- ▶ Teams must use GitHub Actions with Terraform to implement CI/CD workflows that automate infrastructure setup and deployment
 - ▶ This counts for 20% of the Term Project grade
- ▶ Teams will manage Terraform configurations in their GitHub repositories, leveraging GitHub Actions to ensure consistent and repeatable deployments
- ▶ The class activity will provide example GitHub Actions and Terraform configurations to guide students in creating their workflows



Collaborate

Application Lifecycle Mgmt.



Communication & ChatOps



Knowledge Sharing



Build

SCM/VCS



CI



Build



Database Management



Test

Testing



Test Automation
Selenium



Deploy

Deployment



Config Management



Artifact Management



Run

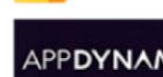
Cloud / IAAS/PAAS



Orchestration



BI/Monitoring/Logging



CI/CD - Challenges

- ▶ **Resistance to Change** – Teams accustomed to traditional development processes may be reluctant to adopt CI/CD due to unfamiliar tools, workflows, or fear of disrupting existing routines
- ▶ **Management Pushback** – Some leaders may push back on CI/CD adoption if its long-term benefits (e.g., stability, automation) don't align with short-term business goals or deadlines
- ▶ **Testing Burden** – Developers must invest time in writing and maintaining automated tests, which can slow early feature development but is essential for long-term reliability
- ▶ **Alert Fatigue** – In large teams, frequent build failures or CI alerts can overwhelm developers, leading to ignored errors and undetected defects
- ▶ **Gradual Adoption, Lasting Impact** – CI/CD implementation takes time and discipline, but it pays off with faster, more stable releases and continuous quality assurance



Build a GitHub Pipeline

- ▶ Do the activity in **Assignments > Activity > GitHub Actions and WordPress**
- ▶ Using GitHub Actions, you are going to take your code from the previous activities and move it to GitHub

